Sistema di prenotazione: guida all'installazione

Guida passo passo per mettere online il modulo `booking` e registrarvi una struttura, con l'hotel come esempio, e per incorporare il form di prenotazione nel sito web esistente della struttura. Destinata allo sviluppatore che si occupa dell'installazione; l'ultima parte elenca cosa resta in carico al gestore.

Un solo deploy serve più strutture: ognuna ha il proprio database, il proprio prefisso negli indirizzi (`/nome-struttura/`) e i propri utenti della dashboard. Il deploy si fa una volta (passi 1 e 2); ogni struttura nuova richiede solo il passo 4 e, per il sito, il passo 5.

Versione del documento: ottobre 2026. Nomi di comandi e file sono in inglese come nel codice.

1. Cosa si ottiene

Al termine dell'installazione:

- Il sito dell'hotel mostra, in una sua pagina, il form di richiesta prenotazione: dati del cliente, date, ospiti, tipologia di camera con disponibilità e prezzo aggiornati in tempo reale.
- Il cliente riceve un'email di riepilogo con un link personale per controllare lo stato o annullare; il gestore riceve la stessa richiesta nella propria casella.
- Il gestore approva o rifiuta dalla dashboard, dove gestisce anche prezzi, periodi e sconti, e registra prenotazioni ricevute per telefono.
- I dati personali dei clienti vengono anonimizzati automaticamente dopo il periodo di conservazione configurato.
- Altre strutture (hotel, ristoranti, impianti sportivi) si aggiungono allo stesso deploy con un comando, senza toccare le altre.

Architettura

Componente	Dove gira	Ruolo
Sito web dell'hotel	hosting attuale, DNS presso l'attuale provider	include il form con una riga di script (iframe)
Worker `booking`	Cloudflare Workers	serve form, dashboard e API di tutte le strutture; invia le email
Direttorio	Cloudflare D1 (`booking-directory`)	elenco delle strutture, loro configurazione, utenti della dashboard
Database della struttura	Cloudflare D1 (`booking-<slug>`, uno per struttura)	richieste, prezzi, società, audit
Email	Brevo (API transazionale)	notifiche a cliente e gestore

Il sito non cambia hosting: il form è una pagina del Worker incorporata in un iframe che si ridimensiona da solo. Nessun dato della prenotazione transita dal sito. I dati di una struttura stanno in un database separato: una struttura non può leggere quelli di un'altra.

Indirizzi

Con `<indirizzo>` l'indirizzo del Worker (passo 2) e `<slug>` il nome breve della struttura scelto al passo 4 (ad esempio `hotel-rossi`):

Indirizzo	Cosa mostra
`https://<indirizzo>/admin`	accesso alla dashboard, unico per tutte le strutture: dopo il login porta alla struttura dell'utente
`https://<indirizzo>/<slug>/`	form pubblico della struttura
`https://<indirizzo>/<slug>/admin`	dashboard della struttura
`https://<indirizzo>/<slug>/privacy`	informativa privacy con i dati del titolare della struttura
`https://<indirizzo>/<slug>/booking/<token>`	pagina personale del cliente (link nelle email)

2. Prerequisiti

- Node.js 24 o superiore e npm, sul computer da cui si pubblica (gli script di gestione importano il codice TypeScript del Worker, cosa che Node fa da solo dalla versione 24).
- Un account Cloudflare (gratuito) con accesso alla sezione Workers & Pages.
- Un account Brevo (gratuito) e la possibilità di aggiungere record DNS al dominio da cui partiranno le email.
- Accesso ai file del sito dell'hotel per aggiungere lo snippet di inclusione.
- La cartella `modules/booking` di questo repository.

3. Passo 1: preparare il deploy (una volta sola)

1 Copia la cartella `modules/booking` in una cartella di lavoro dedicata, ad esempio `booking-platform`, e installa le dipendenze:

```
cd booking-platform
npm install
```

1 Apri `wrangler.jsonc` e sistema tre cose:

Chiave	Valore
`name`	nome del Worker, solo minuscole e trattini, ad esempio `booking`: diventa parte dell'indirizzo pubblico
`vars.PASSWORD_HASH_ITERATIONS`	lascia `20000` sul piano gratuito di Cloudflare; `600000` sul piano Paid (vedi `docs/piano-paid.md`)
`d1_databases`	togli il blocco di esempio `DB_DEMO`: le strutture le aggiunge lo script del passo 4. Resta solo `DIRECTORY`, il cui `database_id` arriva dal passo 2

La configurazione delle strutture (nome, tipologia, email, orari, conservazione dei dati) non sta più in questo file: vive nel direttorio e si scrive con lo script del passo 4.

1 Prova in locale, così da vedere il sistema funzionare prima di toccare Cloudflare. Lo script `tenant:add` con `--local` crea una struttura di prova nei database locali di `wrangler dev` e stampa la password del suo utente:

```
cp .dev.vars.example .dev.vars
npm run migrate:local
npm run tenant:add -- hotel-prova --type hotel --name "Hotel Prova" --username prova --local
npm run dev
```

Apri `http://localhost:8787/admin`, entra con l'utente `prova` e la password stampata: la dashboard si apre su `http://localhost:8787/hotel-prova/admin`. Il form pubblico è su `http://localhost:8787/hotel-prova/`. Le email in locale non partono finché `BREVO_API_KEY` è vuota. Prima di pubblicare, togli dal `wrangler.jsonc` il blocco `DB_HOTEL_PROVA` aggiunto dalla prova.

4. Passo 2: Cloudflare (una volta sola)

4.1 Accesso e direttorio

1 Accedi con wrangler all'account Cloudflare: si apre il browser per autorizzare.

```
npx wrangler login
```

1 Crea il database del direttorio e copia l'identificativo stampato dal comando nel campo `database_id` del blocco `DIRECTORY` di `wrangler.jsonc`:

```
npx wrangler d1 create booking-directory
```

1 Applica lo schema del direttorio:

```
npm run migrate:remote
```

4.2 Secret

I secret non stanno mai in `wrangler.jsonc` né in git. Caricali uno per uno; wrangler chiede il valore in modo interattivo.

```
npx wrangler secret put ADMIN_SECRET
npx wrangler secret put BREVO_API_KEY
```

- `ADMIN_SECRET`: chiave che firma i cookie di sessione di tutte le strutture. Genera una stringa casuale lunga, ad esempio con `node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"`.
- `BREVO_API_KEY`: la otterrai al passo 3. Puoi rimandare questo comando a dopo; finché manca, il sistema funziona ma non invia email.

Non esiste più una password unica della dashboard: ogni struttura ha i propri utenti, creati al passo 4.

4.3 Pubblicazione

```
npm run deploy
```

Al termine wrangler stampa l'indirizzo del Worker, nella forma `https://booking.<sottodominio>.workers.dev`. Il sottodominio `workers.dev` è assegnato all'account e si può rinominare dalla dashboard Cloudflare in Workers & Pages. Annota l'indirizzo: è l'**indirizzo del servizio di prenotazione**, usato nei passi successivi. Il job notturno di anonimizzazione (cron) è attivo automaticamente con il deploy e lavora su tutte le strutture.

Verifica subito:

- `https://<indirizzo>/api/health` risponde `{"ok":true}`.
- `https://<indirizzo>/admin` mostra la pagina di accesso (nessun utente esiste ancora: lo crea il passo 4).

4.4 Dominio personalizzato (facoltativo)

Un indirizzo come `prenota.tuodominio.it` al posto di `workers.dev` è possibile solo se il dominio è gestito dai DNS di Cloudflare: i Custom Domain dei Worker richiedono che la zona sia attiva su Cloudflare e non si possono creare su un hostname che ha già un record CNAME presso un altro provider. Con i DNS presso un altro provider ci sono due strade:

- **Restare su** `workers.dev`**.** Funziona senza altre configurazioni ed è invisibile al cliente finale, perché il form è incorporato nel sito. I punti in cui l'indirizzo compare sono il link personale nelle email e il link di accesso dei gestori. Cloudflare lo descrive come adatto a progetti non critici; è accettabile se si accetta l'indirizzo tecnico nelle email.
- **Spostare i DNS del dominio su Cloudflare** (piano gratuito): si aggiunge il dominio in Cloudflare, si ricopiano i record esistenti, si cambiano i nameserver presso il registrar, poi in Workers & Pages, Worker, Settings, Domains & Routes, si aggiunge `prenota.tuodominio.it` come Custom Domain.

Servendo più strutture, il dominio è quello di chi offre il servizio, non dell'hotel: le strutture si distinguono dal prefisso `/<slug>/`, non dal dominio. Se scegli il dominio personalizzato, da qui in poi usa quello come indirizzo del servizio.

5. Passo 3: Brevo (una volta sola)

Brevo è il servizio che consegna le email. Il Worker lo usa tramite l'API transazionale, in solo testo, con una sola chiave per tutte le strutture. Riferimenti ufficiali: `https://help.brevo.com` (sezione Senders, Domains & dedicated IPs) e `https://developers.brevo.com/reference/sendtransacemail`.

5.1 Mittente: uno per tutti o uno per struttura

Ogni struttura ha il proprio mittente (`email_sender`, passo 4) e la propria casella del gestore (`email_admin`). Il mittente deve stare su un dominio autenticato nell'account Brevo: la strada semplice è un mittente del servizio per tutte le strutture, ad esempio `prenotazioni@tuodominio.it`, con il nome della struttura come nome visualizzato (lo mette il Worker) e la casella del gestore come Reply-To (anche questo lo fa il Worker: rispondere all'email scrive al gestore o al cliente). Un mittente sul dominio dell'hotel (`prenotazioni@nomehotel.it`) è possibile, ma richiede di autenticare anche quel dominio nell'account Brevo con i record DNS dell'hotel.

5.2 Account e autenticazione del dominio

1 Crea l'account su `https://www.brevo.com` e completa l'attivazione richiesta da Brevo per l'invio transazionale.

2 Vai in **Senders, Domains & Dedicated IPs**, scheda **Domains**, e aggiungi il dominio del mittente, ad esempio `tuodominio.it`.

3 Brevo mostra i record DNS da creare presso il provider del dominio: un record TXT di verifica (il "Brevo code"), un record TXT o CNAME per la firma **DKIM** e un record TXT **DMARC** su `_dmarc.tuodominio.it`. Copia nomi e valori esattamente come li mostra Brevo: cambiano per ogni account.

4 Attendi la propagazione (da pochi minuti ad alcune ore) e premi **Authenticate** o **Verify** nella pagina del dominio finché tutti i record risultano verificati.

5 Nella scheda **Senders** aggiungi l'indirizzo scelto come mittente, ad esempio `prenotazioni@tuodominio.it`.

Senza l'autenticazione del dominio le email finiscono facilmente nello spam o vengono rifiutate: non saltare questo passaggio.

5.3 Chiave API

1 In alto a destra apri il menu del profilo e poi **SMTP & API**, scheda **API Keys**.

2 Crea una nuova chiave, ad esempio con nome `booking`, e copiala subito: Brevo non la mostra più.

3 Caricala come secret del Worker:

```
npx wrangler secret put BREVO_API_KEY
```

Il secret è attivo da subito, senza nuovo deploy.

5.4 Prova di invio

Dopo il passo 4, dalla dashboard della struttura registra una prenotazione diretta con un tuo indirizzo email come cliente: devono arrivare l'email di conferma al cliente, con il link personale, e nessuna email al gestore (è stato lui a registrarla). Poi invia una richiesta dal form pubblico: devono arrivare due email, una al cliente e una al gestore. Se non arriva nulla, controlla i log del Worker:

```
npx wrangler tail
```

Un invio fallito lascia una riga `notification_failed` nella tabella `audit_log` della struttura e non blocca mai la prenotazione.

5.5 Limiti del piano gratuito

Diverse fonti indipendenti riportano per il piano gratuito di Brevo un limite di **300 email al giorno**, senza accumulo dei giorni non usati; i piani a pagamento tolgono il limite giornaliero. Il dato non è stato verificato direttamente sulla pagina prezzi di Brevo al momento della scrittura: controllalo su `https://www.brevo.com/pricing/`. Ogni richiesta genera al massimo due email: il limite vale per l'insieme delle strutture, quindi con molte strutture attive va tenuto d'occhio.

6. Passo 4: registrare una struttura

Un comando crea tutto quello che serve a una struttura: il suo database su Cloudflare, il binding in `wrangler.jsonc`, lo schema, la riga nel direttorio e il primo utente della dashboard.

```
npm run tenant:add -- hotel-rossi --type hotel --name "Hotel Rossi" --username rossi --email-sender prenotazioni@t
```

Argomento	Significato
`hotel-rossi`	lo slug: minuscole, cifre e trattini, da 2 a 40 caratteri. È il prefisso degli indirizzi della struttura e non si cambia più
`--type`	`hotel`, `restaurant` oppure `facility` (impianto sportivo)
`--name`	nome della struttura come apparirà in pagine ed email
`--username`	nome utente del gestore, minuscolo
`--email-sender`, `--email-admin`	mittente delle email e casella del gestore (vedi 5.1)
`--privacy-controller`, `--privacy-address`, `--privacy-email`	titolare del trattamento mostrato nell'informativa privacy della struttura
`--lang`	lingua iniziale dell'interfaccia, `it` (default) o `en`
`--retention-days`	giorni di conservazione dei dati del cliente dopo la fine della prenotazione, `90` di default, `0` per non anonimizzare mai
`--embed-origin`	origine del sito della struttura autorizzato a incorporare il form, si può impostare anche dopo (passo 5)

Argomento	Significato
`--remote`	lavora sui database di Cloudflare (`--local` su quelli di `wrangler dev`)

Lo script si ferma se lo slug o il nome utente esistono già, e, sul piano gratuito di Cloudflare, al decimo database dell'account (il direttorio più nove strutture): per andare oltre serve il piano Paid e l'opzione `--allow-many-databases`.

Al termine stampa la **password del gestore, una sola volta**: consegnala insieme al nome utente e al link `https://<indirizzo>/admin`, e chiedi al gestore di cambiarla al primo accesso con il bottone "Cambia password" (almeno 12 caratteri; le password già comparse in violazioni di dati note vengono rifiutate). Poi pubblica: il database nuovo è raggiungibile dal Worker solo dopo il deploy.

```
npm run deploy
```

Le tipologie di camera (nome, ospiti massimi, numero di camere, prezzo base) non si configurano qui: le inserisce il gestore dalla sezione Camere della dashboard (passo 6). La migrazione crea quattro tipologie di esempio ("Camera singola", "Camera doppia", "Camera tripla", "Suite familiare") che il gestore può modificare o eliminare.

Per cambiare in seguito un dato della struttura (nome, email, orari, conservazione, titolare privacy) si aggiorna la sua riga nel direttorio, senza deploy:

```
npx wrangler d1 execute booking-directory --remote --command "UPDATE tenants SET email_admin = 'nuova@hotelrossi.i
```

Le colonne disponibili sono elencate nel README del modulo (sezione "Multi-cliente").

7. Passo 5: incorporare il form nel sito

Il form viene incluso in una pagina del sito tramite un iframe creato dallo script `embed.js` servito dal Worker. L'iframe si adatta in altezza al contenuto, quindi non mostra barre di scorrimento interne.

7.1 Autorizzare il sito

Per motivi di sicurezza il Worker rifiuta di essere incorporato da qualunque pagina, salvo quella del sito autorizzato della struttura. L'origine si scrive nel direttorio (con `--embed-origin` al passo 4, oppure dopo):

```
npx wrangler d1 execute booking-directory --remote --command "UPDATE tenants SET embed_origin = 'https://www.hotel
```

Solo schema e host, senza percorso né barra finale. Se il sito risponde sia con `www` sia senza, scegli la variante a cui reindirizzi le altre. Non serve un nuovo deploy: il Worker legge il direttorio a ogni richiesta. Verifica che `https://<indirizzo>/hotel-rossi/embed` risponda 200: finché l'origine è vuota risponde 404.

7.2 Snippet da inserire nel sito

Nella pagina "Prenota" del sito aggiungi, nel punto in cui deve comparire il form:

```
<div id="booking"></div>
<script src="https://<indirizzo>/embed.js" data-slug="hotel-rossi" data-target="#booking" defer></script>
```

Attributo obbligatorio: `data-slug`, lo slug della struttura. Attributi facoltativi dello script: `data-target` è il selettore del contenitore (default `#booking`), `data-title` il titolo accessibile dell'iframe (default `Booking`).

Se il sito ha una propria Content Security Policy, deve ammettere l'indirizzo del servizio nelle direttive `script-src` (per `embed.js`) e `frame-src` (per l'iframe). Senza CSP non serve nulla.

7.3 Aspetto

Dentro l'iframe il form non mostra testata né piè di pagina: li fornisce la pagina del sito. I colori seguono i token definiti in `public/css/base.css` nel blocco `[data-booking-type="hotel"]` e valgono per tutte le strutture dello stesso tipo: per adattarli modifica quei valori e pubblica di nuovo. Il font è Inter, incluso nel Worker.

La lingua iniziale è quella della struttura (`--lang`); il pulsante in alto nel form permette al cliente di passare a inglese o italiano e la scelta resta memorizzata nel browser. Dentro l'iframe il pulsante non c'è: il form segue l'attributo `lang` dell'elemento `<html>` della pagina del sito, anche quando cambia a runtime, quindi un sito bilingue non deve fare altro che aggiornare quell'attributo.

7.4 Alternativa senza iframe

Se preferisci non incorporare, un semplice link o pulsante "Prenota" che apre `https://<indirizzo>/hotel-rossi/` funziona senza impostare l'origine: la pagina del Worker ha testata e piè di pagina propri.

8. Passo 6: configurazione a carico del gestore

Queste operazioni si fanno dalla dashboard, raggiunta da `https://<indirizzo>/admin` con nome utente e password, e non richiedono lo sviluppatore. La sessione dura 8 ore; dopo dieci password errate l'account resta bloccato per 15 minuti.

1 **Cambia password** (testata della dashboard): al primo accesso, con la password attuale, la nuova e la sua conferma. Cambiarla chiude le altre sessioni aperte con lo stesso utente.

2 **Camere**: con "Nuova tipologia" si crea ogni tipologia di camera con nome, ospiti massimi, numero di camere e prezzo base a notte (obbligatorio); "Modifica" ed "Elimina" sulla riga. Le quattro tipologie di esempio vanno adattate o eliminate. Una tipologia con prenotazioni future (in attesa o confermate) non si può eliminare: prima vanno annullate le prenotazioni. Riducendo il numero di camere sotto le prenotazioni confermate la dashboard avvisa, ma non blocca. La tabella mostra anche le camere libere per le date scelte.

3 **Prezzi**: il prezzo base sta nella tipologia (sezione Camere); qui si aggiungono i periodi stagionali e le eventuali eccezioni, infine gli sconti per durata. Per cambiare il prezzo di un solo giorno è più comodo il calendario Prezzi (sezione Calendario, selettore "Prezzi"): premendo il prezzo di un giorno si apre un popup con il nuovo prezzo, che vale solo per quel giorno anche se il giorno sta dentro un periodo; "Torna al prezzo base" annulla l'eccezione. Il prezzo viene mostrato (e precompilato nella prenotazione diretta) solo se ogni notte del soggiorno ha un prezzo, dal periodo che la copre o dal base. Il prezzo mostrato al cliente riporta la dicitura "Tasse incluse, tassa di soggiorno esclusa".

4 **Notifiche**: le richieste arrivano via email e compaiono nella sezione Notifiche, dove si approvano o rifiutano. Una tipologia risulta esaurita solo quando le prenotazioni approvate coprono tutte le camere.

5 **Nuova prenotazione**: per i clienti che chiamano o scrivono, con gli stessi campi del form; nasce già confermata e il cliente riceve l'email con il link personale. Il prezzo è precompilato con il calcolo e si può sostituire con la cifra concordata.

6 **Pagamenti**: aprendo una prenotazione confermata dal calendario si registra l'acconto ricevuto ("Registra acconto", con l'importo) o il pagamento completo ("Segna come pagata"). Con un acconto il popup mostra il saldo ancora da pagare; al saldo si preme "Segna come pagata". Nel calendario le prenotazioni con acconto e quelle pagate hanno un colore proprio (vedi legenda). L'acconto richiede che la prenotazione abbia un prezzo totale.

7 **Modifica**: dal popup di una prenotazione in attesa o confermata, "Modifica" riapre tutti i campi (dati del cliente, date, camera, ospiti, prezzo, note, lingua). Premendo "Salva" compare il riepilogo dei soli campi

cambiati, con la spunta "Avvisa il cliente via email": la modifica viene salvata solo con "Conferma modifiche". Il prezzo resta quello registrato finché non cambiano date, camera o totale. L'importo già pagato resta registrato: se il nuovo totale è più alto la prenotazione torna "Acconto pagato" con il saldo da pagare (al saldo si preme di nuovo "Segna come pagata"); se è più basso resta "Pagata" e il popup mostra l'importo da rimborsare. Il riepilogo prima della conferma mostra queste conseguenze; l'email al cliente riporta l'importo già pagato e l'eventuale saldo, non il rimborso.

8 **Esporta CSV**: sotto "Nuova prenotazione", scarica un file per Excel con i soggiorni che toccano l'intervallo di date scelto (confermati, rifiutati e annullati; non quelli in attesa), una riga per soggiorno: check-in, check-out, nome, cognome, email, telefono, tipologia, adulti, bambini, prezzo totale concordato, cifra pagata, stato del pagamento e stato della prenotazione. La "cifra pagata" è quanto registrato con "Registra acconto" o "Segna come pagata"; resta vuota se la prenotazione non aveva un prezzo quando è stata segnata pagata. I telefoni con prefisso + compaiono preceduti da un apostrofo (protezione dalle formule di Excel).

Informativa privacy: la pagina `https://<indirizzo>/<slug>/privacy` riporta titolare, indirizzo, email di contatto e giorni di conservazione presi dal direttorio (passo 4): controlla che compaiano i dati giusti; il form rimanda a quella pagina. Il testo è in italiano; se il sito si rivolge a clienti stranieri serve una versione inglese, da aggiungere in `public/privacy.html`.

Ristoranti e impianti sportivi hanno la stessa dashboard con sezioni diverse: niente Camere e Prezzi; l'impianto ha la sezione Società (le organizzazioni che prenotano, con il loro link personale) al posto del form pubblico.

9. Checklist finale

- `https://<indirizzo>/api/health` risponde `{"ok":true}`.
- `https://<indirizzo>/admin` accetta nome utente e password e porta alla dashboard della struttura, con Calendario, Notifiche, Camere e Prezzi.
- Il gestore ha cambiato la password iniziale.
- Le tipologie di camera sono quelle reali dell'hotel, ognuna con il suo prezzo base.
- Il form nel sito mostra le camere disponibili e il prezzo cambiando date e tipologia.
- Una richiesta di prova dal sito produce due email, con il link personale funzionante nell'email del cliente.
- L'approvazione dalla dashboard produce l'email di conferma.
- Il dominio del mittente risulta autenticato in Brevo e le email non finiscono nello spam.
- `https://<indirizzo>/<slug>/privacy` mostra il titolare giusto.
- `.dev.vars` e i secret non sono in git; in `wrangler.jsonc` non restano blocchi di prova (`DB_DEMO`, `DB_HOTEL_PROVA`).

10. Manutenzione

Operazione	Come
Cambiare tipologie di camera, quantità, prezzi, periodi, sconti	dalla dashboard, nessun deploy

Operazione	Come
Cambiare nome, email, orari, conservazione, titolare privacy, origine del sito di una struttura	`UPDATE tenants ...` sul direttorio (passo 4), nessun deploy
Aggiungere una struttura	`npm run tenant:add -- ... --remote`, poi `npm run deploy`
Sospendere una struttura (tutti i suoi indirizzi rispondono 404, i dati restano)	`UPDATE tenants SET status = 'suspended' WHERE slug = '...';` `'active'` per riattivarla
Aggiornare il codice	`npm run migrate:remote` se ci sono nuove migrazioni (le applica a tutti i database), poi `npm run deploy`
Backup di una struttura	`npx wrangler d1 export booking-<slug> --remote --output backup-<slug>-AAAAMMGG.sql` (il file contiene dati personali: non committarlo); il direttorio con `booking-directory`
Vedere i log in tempo reale	`npx wrangler tail`
Gestore che ha dimenticato la password, o account bloccato	`npm run tenant:password -- <username> --remote`: nuova password stampata una volta, account sbloccato, sessioni chiuse
Invalidare tutte le sessioni di tutte le strutture	`npx wrangler secret put ADMIN_SECRET` con un nuovo valore

Fai un backup prima di ogni `migrate:remote`.

11. Costi e limiti

Il sistema è tarato sul piano gratuito di Cloudflare: cosa cambia passando al piano Paid (5 dollari al mese), e quando conviene, è in `docs/piano-paid.md`. Valori dei piani gratuiti al momento della scrittura, da ricontrollare sulle pagine ufficiali prima di decidere.

Servizio	Piano gratuito	Fonte
Cloudflare Workers	100.000 richieste al giorno, 10 ms di CPU per richiesta, 5 cron per account	`https://developers.cloudflare.com/workers/platform/limits/`
Cloudflare D1	10 database per account (il direttorio più nove strutture), 500 MB ciascuno, 5 milioni di righe lette e 100.000 scritte al giorno in totale	`https://developers.cloudflare.com/d1/platform/limits/`
Brevo	300 email al giorno in totale (dato di fonti terze, da verificare)	`https://www.brevo.com/pricing/`

Per poche strutture di piccole o medie dimensioni questi limiti sono ampiamente sufficienti: una richiesta costa poche richieste HTTP e qualche riga letta. Il limite che si incontra per primo è quello dei dieci database.

12. Riferimenti

- Cloudflare, Custom Domains dei Worker: `https://developers.cloudflare.com/workers/configuration/routing/custom-domains/`
- Cloudflare, sottodominio workers.dev: `https://developers.cloudflare.com/workers/configuration/routing/workers-dev/`

- Cloudflare, comandi wrangler: `https://developers.cloudflare.com/workers/wrangler/commands/`
- Cloudflare, limiti e prezzi D1: `https://developers.cloudflare.com/d1/platform/limits/`, `https://developers.cloudflare.com/d1/platform/pricing/`
- Brevo, API transazionale: `https://developers.brevo.com/reference/sendtransacemail`
- Brevo, centro assistenza (autenticazione del dominio, chiavi API): `https://help.brevo.com`
- README del modulo: `modules/booking/README.md`; passaggio al piano Paid: `modules/booking/docs/piano-paid.md`